GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 3 OF 10 | CORE TOPIC

Trees and Tries

22 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	17	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Trees and Tries | 22 questions | tap a title to jump to it

E	1. Find right-side view of binary tree	*Trees & BST*	**PracHub**	**2025-11-01**
M	2. URL Request Handler Router	*String*	**1Point3Acres**	**2026-10-16**
M	3. Expression Tree Single-Leaf Mutation	*Tree*	**1Point3Acres**	**2026-09-27**
M	4. Compute File System Total Size	*Tree*	**1Point3Acres**	**2026-09-02**
M	5. Nested List Weight Sum (LC 339) with Follow-Ups	*Tree*	**1Point3Acres**	**2026-08-16**
M	6. Longest Consecutive Path in Binary Tree	*Tree*	**1Point3Acres**	**2026-07-02**
M	7. Trie Prefix Search / Autocomplete	*Trie*	**1Point3Acres**	**2026-07-02**
M	8. Greedy Longest-Match Dictionary Tokenizer	*String*	**1Point3Acres**	**2026-06-15**
M	9. Employee Hierarchy Add / Delete	*Tree*	**1Point3Acres**	**2026-06-10**
M	10. Check if all substrings are dictionary words	*Trie*	**PracHub**	**2026-03-11**
M	11. Return dictionary words matching a prefix	*Trie*	**PracHub**	**2026-03-09**
M	12. Streaming Insert + Find K-th Largest	*Data structure*	**1Point3Acres**	**2026-02-04**
M	13. Solve chat, grid paths, and car rentals	*Trie*	**PracHub**	**2025-12-02**
M	14. Find mode in a trinary search tree	*Trees & BST*	**PracHub**	**2025-11-16**
M	15. Solve tree, graph, sliding-window problems	*Trees & BST*	**PracHub**	**2025-08-04**
M	16. Implement zigzag level-order traversal	*Trees & BST*	**PracHub**	**2025-08-01**
M	17. Lowest Common Ancestor of a Binary Tree III	*Trees & BST*	**HackTheRounds**	**no date**
M	18. Lowest Common Ancestor	*Trees & BST*	**HackTheRounds**	**no date**
H	19. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	*Trees & BST*	**PracHub**	**2026-09-16**
H	20. Huffman Tree Construction	*Tree*	**1Point3Acres**	**2026-06-01**
H	21. Compute distance-sum from every tree node	*Trees & BST*	**PracHub**	**2026-01-22**
H	22. Implement Longest-Match Text Replacement	*Trie*	**PracHub**	**2026-01-20**

STUDY HERE: topic 3, Trees and Tries

EASY | IN APP LC 199 | PracHub | Technical Screen | 1 of 22

Find right-side view of binary tree

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK ALL TIME 34.9 30 DAYS 0 PRIORITY 111/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Posted on PracHub: 2025-11-01 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode righ
t; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

APPROACH

BFS taking the last node of each level, or DFS visiting right before left and recording the first node seen at each depth. O(n).

HOW TO THINK ABOUT IT

The right side view is the last node of every level, or equivalently the first node you reach at each depth if you visit right children before left. Either way you need the depth of each node, so carry it in the traversal.

Basis: LeetCode 199, Binary Tree Right Side View.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def right_side_view(root):
    levels = level_order(root)  # list of lists
    return [level[-1] for level in levels]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION · DFS visiting right before left, first node at each depth

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

FOLLOW-UP 2 Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

FOLLOW-UP 3 Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Find right-side view of binary tree
#
# You are given the root of a binary tree. From the
# **right side** of the tree, at each depth you can see
# exactly one node: the rightmost node at that depth.
# Write a function that returns a list of the values of
# the nodes visible when looking at the tree from the
# right side, ordered from **top to bottom**. --- ###
# Function Signature (one possible form) text
# rightSideView(root: TreeNode) -> List[int] Where
# TreeNode is defined as: text class TreeNode { int val;
# TreeNode left; TreeNode right; } --- ### Example
# Consider this binary tree: text 1 / \ 2 3 \ \ 5 4 The
# right-side view is: text [1, 3, 4] Explain the algorithm
# you would use, including its time and space complexity.
# You may use either breadth-first search (BFS) or depth-
# first search (DFS).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Left side view, or both.
# 2. Avoid recursion.
# 3. Top view or bottom view.
#
# INTERVIEW NOTES
# The right side view is the last node of every level, or
# equivalently the first node you reach at each depth if
# you visit right children before left. Either way you
# need the depth of each node, so carry it in the
# traversal.
#
# BRUTE FORCE (say it first): Collect every level, then
# take the last of each
# Build the full list of nodes per level with a queue and
# return the last value of each list. It stores every node
# of the tree when only one per level is needed. Time
# O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Collect every level, then
# take the last of each | time O(n)
def right_side_levels(root):
    if not root:
```

```python
        return []
    levels, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    return [lv[-1] for lv in levels]


# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
        if node is None:
```

```python
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out


# FOLLOW-UP 1: Left side view, or both.
# Answer: Swap the order of the two recursive calls. For
# both views record the first and last node per depth in
# one pass.
def left_and_right_views(root):
    # both views in one pass: first and last node per
    # level
```

```python
    left, right = [], []
    queue = deque([root] if root else [])
    while queue:
        left.append(queue[0].val)
        right.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return left, right


# FOLLOW-UP 2: Avoid recursion.
# Answer: Use the BFS version and keep only the last node
# per level (no per-level list), or an explicit stack of
# (node, depth) pushing left before right so that right is
# popped first.
def right_side_view_bfs(root):
    # no recursion: keep only the LAST node of every level
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out


# FOLLOW-UP 3: Top view or bottom view.
# Answer: Track a horizontal distance per node (left minus
# one, right plus one) and keep the first (top) or last
# (bottom) node seen per distance in BFS order.
def top_and_bottom_view(root):
    # horizontal distance: left child -1, right child +1
    top, bottom = {}, {}
    queue = deque([(root, 0)] if root else [])
    while queue:
        node, hd = queue.popleft()
        # the first node seen at a distance is the top
        # one;
        # the last one seen is the bottom one
        top.setdefault(hd, node.val)
```

```python
        bottom[hd] = node.val
        if node.left:
            queue.append((node.left, hd - 1))
        if node.right:
            queue.append((node.right, hd + 1))
    order = sorted(top)
    return ([top[h] for h in order],
            [bottom[h] for h in order])
```

```
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   3
    #    \    \
    #     5    4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
                    TreeNode(3, None, TreeNode(4)))
    assert right_side_view(tree) == [1, 3, 4]
    assert right_side_view_bfs(tree) == [1, 3, 4]
    views = left_and_right_views(tree)
    assert views == ([1, 2, 5], [1, 3, 4])
    assert right_side_view(None) == []
    # a left-only tree is still seen from the right
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert right_side_view(chain) == [1, 2, 3]
    top, bottom = top_and_bottom_view(tree)
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]
```

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[int]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
`([1, 2, 3, None, 5, None, 4],)`	`[1, 3, 4]`
`([1, 2, 3, 4],)`	`[1, 3, 4]`

STUDY HERE: topic 3, Trees and Tries

Site tags: string, parsing, data-structure, trie, dfs

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen, onsite-coding | 2 of 22

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 222/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup
    for /users/123/photo/456 matches that registered
    handler.
```

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

BRUTE FORCE TO SAY OUT LOUD

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

OPTIMAL IDEA

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

LIKELY FOLLOW-UPS

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

PITFALLS

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L)	**O(1)**
per lookup	extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching)	**O(total pattern length)**
per lookup, O(total pattern length) to build	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: URL Request Handler Router
#
# Examples: Register /users/{userId}/photo/{photoId}. A
# lookup for /users/123/photo/456 matches that registered
# handler.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.
#
# BRUTE FORCE (say it first): Scan every registered
# pattern
# Compare the URL's segments with each pattern's segments
# (same count, each one equal or a placeholder), collect
# the matches and keep the best by specificity. It is the
# correctness baseline, and a good oracle for the trie.
# Time O(H * L) per lookup. Space O(1) extra.


# HELPERS for the brute force
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# BRUTE FORCE (say it first): try every route in a list
def fits(pat, segs):
    # the placeholder values if the route fits, else None
    params = {}
    for i, p in enumerate(pat):
        if p == "*":
            return params if len(segs) > i else None
        if i >= len(segs):
            return None
        if p.startswith("{") and p.endswith("}"):
            params[p[1:-1]] = segs[i]
        elif p != segs[i]:
            return None
    return params if len(pat) == len(segs) else None


class RouterBrute:
    def __init__(self):
        self.routes = []

    def add(self, pattern, handler=None, method="GET"):
        pat = [method] + segments(pattern)
        self.routes.append((pat, handler or pattern))

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        best = None
        for pat, handler in self.routes:
            params = fits(pat, segs)
            if params is None:
                continue
            # literal beats placeholder beats trailing *
            score = [2 if p == s else 1 if p[0] == "{"
                     else 0 for p, s in zip(pat, segs)]
            if best is None or score >= best[0]:
                best = (score, handler, params)
        return (best[1], best[2]) if best else (None, {})


# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
```

```
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
# FOLLOW-UP 1: Different HTTP methods per route.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
```

```
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
```

```
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)


if __name__ == "__main__":
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
    # the brute force agrees with the trie
    import random
    words = ["a", "b", "{}", "*"]
    for _ in range(300):
        fast, slow = Router(), RouterBrute()
        for k in range(random.randint(1, 5)):
            n = random.randint(1, 3)
            parts = [random.choice(words)
                     for _ in range(n)]
            if "*" in parts:
```

```
                parts = parts[:parts.index("*") + 1]
            # a placeholder is named after its position
            parts = ["{p%d}" % i if p == "{}" else p
                     for i, p in enumerate(parts)]
            pattern = "/" + "/".join(parts)
            fast.add(pattern, f"h{k}")
            slow.add(pattern, f"h{k}")
        for _ in range(8):
            n = random.randint(1, 3)
            url = "/" + "/".join(random.choice("abc")
                                 for _ in range(n))
            assert fast.match(url) == slow.match(url), url
```

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.
The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.
Matching approach
Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.
A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.
Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.
For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization

and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, dfs, simulation, medium, state-machine

MEDIUM | 1Point3Acres | onsite-coding | 3 of 22

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 203/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

BRUTE FORCE TO SAY OUT LOUD

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

OPTIMAL IDEA

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

DRY-RUN THESE CASES

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

LIKELY FOLLOW-UPS

Several flips applied together, or an undo.

INTERVIEW-DAY PLAN

PITFALLS

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# Evaluate a boolean expression tree, then answer the root
# value after each single-leaf flip. Current variants
# include XOR/OR/AND/NOT operators, no parent pointer on
# the node interface, and an expectation that the
# candidate cache parent links or paths during the initial
# traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.
#
# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back
# For each of the L leaves, toggle it, evaluate the entire
# tree, and toggle it back. It is the correct baseline and
# the oracle for the cached version. Time O(n^2). Space
# O(h).


# HELPERS for the brute force
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()
```

```python
def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(
        node.op,
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out


# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back | time O(n^2)
class XNode:
    def __init__(
        self, op=None, left=None, right=None, val=None
    ):
        self.op, self.left, self.right, self.val = (
            op,
            left,
            right,
            val,
        )


def flips_brute(root):
    out = []
    for leaf in _leaves(root):
        leaf.val = not leaf.val
        out.append(_eval_full(root))
        leaf.val = not leaf.val
    return out


# HELPERS for the base question
```

```python
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
```

```
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
```

```
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers


# FOLLOW-UP 3: Which leaves change the answer at all?
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
```

```
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found


# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


if __name__ == "__main__":
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
```

```
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.
The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.
Clarify whether XOR is binary-only and whether the tree is guaranteed valid.
Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.
Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, dfs, medium

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 4 of 22

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 204/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees. Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

INTERVIEW-DAY PLAN

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute File System Total Size
#
# Examples: root = Dir( Dir(File(10), File(20)), File(5),
# Dir(File(30)), ) Total: 65
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
#
# BRUTE FORCE (say it first): Plain recursion
# Sum a file's size, or the sum over a directory's
# children. Correct for ordinary trees and the answer most
# interviews want first. It overflows on very deep trees
# and loops forever on a cycle. Time O(n). Space O(depth)
# call stack.


import heapq


# HELPERS for the brute force
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BRUTE FORCE (say it first): Plain recursion | time O(n)
def total_size_recursive(node):
    if isinstance(node, File):
        return node.size
    return sum(
        total_size_recursive(c) for c in node.children
    )
```

```
# HELPERS for follow-up 3
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
```

```
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


# FOLLOW-UP 1: The tree is queried often and files change.
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
```

```
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
```

```
            stack.extend(node.children)
    return out


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))


if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, recursion, parsing, medium

MEDIUM | 1Point3Acres | onsite-coding | 5 of 22

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 16.5** | **30 DAYS 0** | **PRIORITY 156/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×
2
# + 14×2 + 2×2 + 91×3 = 320. Use [2, [91]] as a manual
# trace for bracket depth and the final-number flush.
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).
#
# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth
# json.loads turns the text into nested lists; a recursive
# function adds each integer times its depth. Clear, but
# it builds the whole structure and recurses as deep as
# the nesting. Time O(n). Space O(depth).
```

```
import json


# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth | time O(n)
def depth_sum_parsed(s):
    def go(items, depth):
        return sum(
            (
                x * depth
                if isinstance(x, int)
                else go(x, depth + 1)
            )
            for x in items
        )
```

```
    return go(json.loads(s), 1)


# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
```

```
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

```python
# FOLLOW-UP 2: What is the inverse version (deeper
# integers weigh less)?
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
```

```python
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, dfs, recursion, medium

MEDIUM | 1Point3Acres | phone-screen | 6 of 22

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 157/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

APPROACH

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether length counts nodes or edges, whether the path goes only parent to child (base) or can bend through a node (follow-up), and whether the sequence must increase by exactly one.

BRUTE FORCE TO SAY OUT LOUD

From every node walk down both children recording the longest run. O(n^2) on a skewed tree.

OPTIMAL IDEA

Base: DFS passing the current run length; it grows by one when the child is exactly the parent plus one, otherwise it resets to 1; keep the global best. O(n). Follow-up: each node returns the longest increasing and longest decreasing chain going down from it; the best path through a node is inc + dec - 1.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A single node, an empty tree, duplicate values, a split branch, a descending chain.

LIKELY FOLLOW-UPS

The any-direction version (above), and an iterative version for very deep trees.

PITFALLS

Counting edges when the interviewer expects nodes. Resetting the run to 0 instead of 1.

HOW TO THINK ABOUT IT

Each node needs to know the longest run that ends at its parent, so pass the current length down and reset it to 1 when the value is not parent + 1. For the bending version each node must report two lengths upward (increasing and decreasing chains going down), and the best path through a node combines one of each.

Basis: LeetCode 298 (parent-to-child) and LeetCode 549 (any direction), Binary Tree Longest Consecutive Sequence I and II.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# parent-to-child path whose values increase consecutively
# by 1. Follow-up: allow the path to extend in any
# direction through the tree, not only from parent to
# child.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.
#
# BRUTE FORCE (say it first): Start a downward walk from
# every node
# For each node, walk down both children while the next
# value is exactly one more, tracking the longest run.
# Correct and the oracle for the base question; nodes deep
# in a chain are re-walked from every ancestor. Time O(n *
# h). Space O(h).


# BRUTE FORCE (say it first): Start a downward walk from
# every node | time O(n * h)
def longest_consecutive_brute(root):
    best = 0

    def down(node):
        a = b = 0
        if node.left and node.left.val == node.val + 1:
            a = down(node.left)
        if (
            node.right
            and node.right.val == node.val + 1
        ):
            b = down(node.right)
        return 1 + max(a, b)

    def visit(node):
        nonlocal best
        if node:
            best = max(best, down(node))
            visit(node.left)
            visit(node.right)

    visit(root)
    return best


# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best


# FOLLOW-UP 4: The path may go in any direction (LeetCode
```

```
# 549).
# Answer: Return the pair (longest increasing chain down,
```

```
# longest decreasing chain down) from each node, and
# update the global best with inc + dec - 1 at the node.
# Verified against exhaustive path enumeration for small
# trees.
def longest_any_direction(root):
    # LeetCode 549: the path may bend at one node
    if root is None:
        return 0
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    # inc[x] / dec[x]: longest chain going down from x
    inc, dec, best = {}, {}, 1
    # children are processed before their parents
    for node in reversed(order):
        up = down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                up = max(up, inc[child] + 1)
            if child.val == node.val - 1:
                down = max(down, dec[child] + 1)
        inc[node], dec[node] = up, down
        # the bend joins both chains (counted node once)
        best = max(best, up + down - 1)
    return best


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   0
    #  /
    # 3
```

```
    left = TreeNode(2, TreeNode(3))
    tree = TreeNode(1, left, TreeNode(0))
    assert longest_consecutive(tree) == 3
    assert longest_consecutive(tree, True) == [1, 2, 3]
    # bending: 2 - 1 - 0 or 3 - 2 - 1 - 0 is length 4
    assert longest_any_direction(tree) == 4
    assert longest_consecutive(None) == 0
    assert longest_any_direction(None) == 0
    # duplicates never extend a chain
    dup = TreeNode(5, TreeNode(5), TreeNode(5))
    assert longest_consecutive(dup) == 1
    assert longest_any_direction(dup) == 1
    # a long chain needs no recursion
    chain = TreeNode(0)
    cur = chain
    for i in range(1, 5000):
        cur.left = TreeNode(i)
        cur = cur.left
    assert longest_consecutive(chain) == 5000
```

NOTES FROM CANDIDATES

In the base version, carry the current consecutive length while traversing downward and reset to 1 whenever the child is not exactly parent + 1.
For the any-direction follow-up, track increasing and decreasing chain lengths at each node, then combine through the node when the left/right child values support a consecutive bridge.
Clarify whether the answer is measured in nodes or edges before coding; Google candidates usually state node count.

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY HERE: topic 3, Trees and Tries

Site tags: trie, tree, dfs, medium

MEDIUM | IN APP LC 208 | 1Point3Acres | tech-screen, onsite-coding | 7 of 22

Trie Prefix Search / Autocomplete

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 36** **30 DAYS 0** **PRIORITY 108/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: your app's interview approach (LeetCode #208)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Frequency: Medium | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

QUESTION

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding.

Build a trie from a dictionary / array of words.
Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.
Some interviewers phrase the query as "a few letters" or "substring"; clarify whether they truly mean arbitrary substring search or the standard prefix/autocomplete contract before coding.
Common follow-ups: deletion, fuzzy match (one-character edit), autocompletion ranking by frequency, and returning only the best K completions.

EXAMPLES

```
Dict ["apple","app","apricot","banana"], prefix "ap"
    -> ["app","apple","apricot"].
```

APPROACH

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

HOW TO THINK ABOUT IT

A trie shares prefixes, so walking down the query prefix costs O(length of the prefix) no matter how many words exist. Everything under that node is a completion. For autocomplete you either collect the words below the prefix node with a DFS (visiting children in sorted order gives sorted output) or store the best few words at every node while inserting.

Basis: LeetCode 208, Implement Trie, plus LeetCode 1268, Search Suggestions System (top completions per prefix).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def search_prefix(words, prefix):
    found = []
    for w in words:
        if w.startswith(prefix):
            found.append(w)
    return sorted(found)
```

TIME	SPACE
O(n * L + n log n) per query	**O(output)**

WHY IT IS SLOW

Every query reads the whole dictionary. A trie reaches the prefix in O(length of the prefix) and reads only the completions.

OPTIMAL SOLUTION · Trie with a DFS over sorted children

TIME	SPACE
O(L + output) per query after O(total characters) to build	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

FOLLOW-UP 2 Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Trie Prefix Search / Autocomplete
#
# LeetCode 208 + 1268. Build a trie from a dictionary and
# return all words sharing a query prefix. Standard Google
# onsite coding. Build a trie from a dictionary / array of
# words. Support searchPrefix(prefix) -> list[str]: return
# all words that start with the query prefix,
```

```
# lexicographically sorted unless ranking is specified.
# Some interviewers phrase the query as "a few letters" or
# "substring"; clarify whether they truly mean arbitrary
# substring search or the standard prefix/autocomplete
# contract before coding. Common follow-ups: deletion,
# fuzzy match (one-character edit), autocompletion ranking
# by frequency, and returning only the best K completions.
# Examples: Dict ["apple","app","apricot","banana"],
# prefix "ap" -> ["app","apple","apricot"].
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best K completions ranked by
# frequency.
# 2. Allow one wrong character (fuzzy match).
#
# INTERVIEW NOTES
# A trie shares prefixes, so walking down the query prefix
# costs O(length of the prefix) no matter how many words
# exist. Everything under that node is a completion. For
# autocomplete you either collect the words below the
# prefix node with a DFS (visiting children in sorted
# order gives sorted output) or store the best few words
# at every node while inserting.
#
# BRUTE FORCE (say it first): Filter the whole word list
# Scan every word, keep those starting with the prefix and
# sort them. Correct and the oracle; every query pays for
# the whole dictionary. Time O(n * L + n log n) per query.
# Space O(output).


# BRUTE FORCE (say it first): Filter the whole word list |
# time O(n * L + n log n) per query
def prefix_words_brute(words, prefix):
    return sorted(
        {w for w in words if w.startswith(prefix)}
    )
```

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []


# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
                out.append(text)
            for ch in n.children:
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out


if __name__ == "__main__":
    t = Trie(k=2)
    for w in ["apple", "app", "apricot", "banana"]:
        t.insert(w)
    ap = t.search_prefix("ap")
    assert ap == ["app", "apple", "apricot"]
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == [
        "app", "apple", "apricot", "banana"]
    # frequency ranking: app was typed three more times
    t.insert("app", 3)
    assert t.suggest("ap") == ["app", "apple"]
    assert t.suggest("zzz") == []
    # one wrong letter in the prefix is allowed
    assert "banana" in t.fuzzy("vana")
    assert t.fuzzy("zzzz") == []
```

NOTES FROM CANDIDATES

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

PREPARATION

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY HERE: topic 3, Trees and Tries

Site tags: string, trie, hashmap, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 8 of 22

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 215/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

APPROACH

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary key that is a prefix of the remaining text. Putting the keys in a trie lets you walk the text character by character from that position and remember the deepest end-of-key node you passed, instead of trying every length. If the tie-break rule changes (priority instead of length), store the priority at the terminal node and keep the best terminal seen.

Basis: Written for this question (longest-prefix matching with a trie, as in LeetCode 208 and the maximal-munch rule of lexers).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L)	**O(dictionary)**
lookups, each O(L)	

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	**O(total key length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.
#
# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length
# At every position try the prefix lengths from longest to
# shortest in a dictionary lookup. Correct and easy; each
# lookup hashes a substring, so it costs O(L^2) per
# position. Time O(n * L) lookups, each O(L). Space
# O(dictionary).


# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length | time O(n * L) lookups, each O(L)
def tokenize_brute(text, table):
    out, i, longest = (
        [],
        0,
        max(map(len, table), default=0),
    )
    while i < len(text):
        for k in range(
            min(longest, len(text) - i), 0, -1
        ):
            if text[i : i + k] in table:
                out.append(table[text[i : i + k]])
                i += k
                break
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# FOLLOW-UP 1: The dictionary is huge.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
```

```
                break
        else:
            out.append(text[i])
            i += 1
    return out


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]
```

NOTES FROM CANDIDATES

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, hashmap, object-design, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 9 of 22

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 227/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

APPROACH

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what happens when a manager is deleted (promote a report, reassign to the grandparent, or reject), what happens when the CEO is deleted, whether adding can create a cycle, and how seniority is defined.

BRUTE FORCE TO SAY OUT LOUD

Store parent pointers only and search the whole tree to find children on every delete. O(n).

OPTIMAL IDEA

Keep id to node, and for each node a parent pointer and a children set. Add: validate the manager exists and the new id is free. Delete: take the children of the removed node and attach them to a replacement (the deleted node's parent, or a chosen successor), update pointers, and remove it from its parent's children. CEO needs a stated policy. Compare candidates by (level descending, insertion order ascending) for the seniority rule.

DRY-RUN THESE CASES

Delete a leaf, a middle manager, the CEO, an unknown id, adding with a missing manager.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Cycle detection by walking up parent pointers, bulk deletes, or a history of changes.

PITFALLS

Leaving dangling children references. Not choosing a policy for the root.

HOW TO THINK ABOUT IT

A hierarchy is a tree you update, so store links in both directions (parent per node, children per node). Then every operation touches only the neighbours of one node instead of searching the tree. For each delete case (leaf, middle, root) say the policy first, because the data structure is easy once the rule is fixed.

Basis: Written. Assumed policies (confirm them): deleting a manager hands their reports to that manager's own manager; deleting the CEO is allowed only if exactly one report exists (that report becomes CEO), otherwise it is rejected; ids are unique.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	**O(n)**

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Design an employee hierarchy class that supports adding
# employees under managers and deleting employees by id.
# Follow-ups cover deleting managers, deleting the CEO,
# cycle handling, and choosing a replacement manager by
# seniority.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.
#
# BRUTE FORCE (say it first): Store only parent pointers
# and search for children
# Keep a dict from employee to manager. To delete, scan
# all entries to find the reports and reassign them.
# Correct, but every delete is a full scan. Time O(n) per
# delete. Space O(n).


# BRUTE FORCE (say it first): Store only parent pointers
# and search for children | time O(n) per delete
class OrgBrute:
    def __init__(self, ceo):
        self.manager = {ceo: None}

    def add(self, emp, mgr):
        if (
            emp in self.manager
            or mgr not in self.manager
        ):
            return False
        self.manager[emp] = mgr
        return True
```

```
    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        reports = [
            e
            for e, m in self.manager.items()
            if m == emp
        ]
        if mgr is None and len(reports) != 1:
            return False
        for r in reports:
            self.manager[r] = mgr
        del self.manager[emp]
        return True


# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# FOLLOW-UP 1: Support moving an employee under another
# manager and reject cycles.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
```

```
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
```

```
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
```

```
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True


if __name__ == "__main__":
    o = Org("ceo")
    for e, m in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(e, m)
    assert not o.add("a", "ceo")
    assert not o.add("x", "nobody")
    # deleting a manager: reports go to the grandparent
    assert o.delete("a")
    assert o.manager["b"] == "ceo"
    assert o.manager["c"] == "ceo"
    # a move that would create a cycle is rejected
    o.add("d", "b")
    assert not o.move("b", "d") and o.move("d", "c")
    # promote the most senior report instead
    p = Org("ceo")
    for e, m in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        p.add(e, m)
    assert p.delete("m", "senior")
    assert p.manager["x"] == "ceo"
    assert p.manager["y"] == "x"
    # the CEO leaves: the most senior report takes over
    q = Org("ceo")
    q.add("a", "ceo")
    q.add("b", "ceo")
    assert q.delete("ceo")
    assert q.root == "a" and q.manager["b"] == "a"
    assert not Org("solo").delete("solo")
```

NOTES FROM CANDIDATES

Use maps from id to node and id to children; keep parent pointers so deletes do not require a full traversal.
CEO deletion needs an explicit policy: reject, promote a child, or create a new root. State the policy and keep it consistent.
For the seniority follow-up, maintain insertion order on nodes and compare (level desc, insertion_order asc).

PREPARATION

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY HERE: topic 3, Trees and Tries

MEDIUM | PracHub | Technical Screen | 10 of 22

Check if all substrings are dictionary words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 82/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: No direct match; related LC 139 Word Break

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

APPROACH

Enumerate all substrings of length >= 3 and look each up in the set; exit on the first failure. Prune: if any length-3 window fails return false immediately. O(n^2) lookups.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the dictionary access (a set or a trie), the maximum length of s, and whether case matters. A string of length n has about n^2 / 2 substrings of length at least 3, and each is a separate requirement.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every substring of length at least 3 and look it up. O(n^2) lookups, each costing O(length) to hash, so O(n^3) characters.

OPTIMAL IDEA

Exit on the first failure; checking all length-3 windows first prunes most inputs. To avoid the length factor, walk a trie: for each start i descend the trie along s[i:], and at every depth of at least 3 require the end-of-word flag; if the path breaks, return false. That is O(n^2) node steps in total.

DRY-RUN THESE CASES

A string shorter than 3 (true vacuously), cats (needs cat, ats, cats), a failure on the first window, a failure only on the longest substring.

LIKELY FOLLOW-UPS

Anagram version (next question), or return the first failing substring.

PITFALLS

Checking only the whole string. Treating a string shorter than 3 as false.

HOW TO THINK ABOUT IT

Every substring is a separate requirement, so no clever skipping exists: you must test about n^2 / 2 substrings. What you can improve is the cost per test and the early exit. A trie lets you extend a start position one character at a time without rehashing the whole substring, and the first failure ends everything.

Basis: Written; every contiguous substring of length >= 3 must be a dictionary word. The dictionary is a set (optionally preprocessed into a trie).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, dictionary):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            if s[i:j] not in dictionary:
                return False
    return True
```

TIME	SPACE
O(n^2)	**O(1)**
lookups, O(n^3) characters hashed	extra

WHY IT IS SLOW

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION · Walk a trie from each start position

TIME	SPACE
O(n^2)	**O(dictionary size)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

FOLLOW-UP 2 The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

FOLLOW-UP 3 The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. **Cost:** still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are dictionary
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** is a valid dictionary word (i.e., for all i, j
# with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in
# dict). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - You may assume
# dictionary lookups are available (e.g.,
# dict.contains(word)), or you can preprocess the
# dictionary. Example: - If s = "cats", you must check
# "cat", "ats", and "cats" (and any other substrings of
# length >= 3).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first failing substring.
# 2. The string is very long.
# 3. The dictionary is huge and sharded.
#
# INTERVIEW NOTES
# Every substring is a separate requirement, so no clever
# skipping exists: you must test about n^2 / 2 substrings.
# What you can improve is the cost per test and the early
# exit. A trie lets you extend a start position one
# character at a time without rehashing the whole
# substring, and the first failure ends everything.
#
# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set
# Slice s[i:j] for every i, j with j - i >= 3 and test
# membership. Correct and simple; every lookup re-hashes
# the whole substring. Time O(n^2) lookups, O(n^3)
# characters hashed. Space O(1) extra.


# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set | time O(n^2) lookups, O(n^3)
# characters hashed
def all_substrings_words_brute(s, words):
    return all(
        s[i:j] in words
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
# FOLLOW-UP 1: Return the first failing substring.
# Answer: Same loops, but return the substring instead of
# False, which is what a user wants to see. Check the
# shorter ones first so the report is the smallest
# counterexample.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None


# SAME CHECK with a trie
class Trie:
    """The same check with a trie."""

    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
            node["$"] = True

    def all_substrings(self, s, min_len=3):
        for i in range(len(s)):
            node = self.root
            for depth, ch in enumerate(s[i:], 1):
                node = node.get(ch)
                if node is None:
                    # no word has this prefix: fails if
                    # the
                    # substring is long enough to count
                    return len(s) - i < min_len
                if depth >= min_len and "$" not in node:
                    return False
        return True
```

```
if __name__ == "__main__":
    words = {"aaa", "aaaa", "aaaaa"}
    assert all_substrings_words("aaaaa", words)
    assert Trie(words).all_substrings("aaaaa")
    # "aab" is a window that is not a word
    assert first_failing("aaab", words) == "aab"
    assert first_failing("aab", words) == "aab"
    assert not all_substrings_words("aab", words)
    assert not Trie(words).all_substrings("aab")
    # shorter than 3 letters: nothing to check
    assert all_substrings_words("ab", set())
    assert Trie([]).all_substrings("ab")
    # longer than every word: fails immediately
    assert first_failing("aaaaaa", words) == "aaaaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```
solution('cat', ['cat'])
```

Test cases

Input	Expected
('cat', ['cat'])	True
('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])	True

STUDY HERE: topic 3, Trees and Tries

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 11 of 22

Return dictionary words matching a prefix

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 106/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a list of strings **`words`** (a dictionary) and a string **`prefix`**. Return all words in **`words`** that start with **`prefix`**. Clarifications/constraints:

- Matching is case-sensitive.
- If no words match, return an empty list.
- If **`words`** contains duplicates, include duplicates in the output.
- You may return the result in any order unless the interviewer asks for a specific ordering (e.g., lexicographic).

APPROACH

Sort once and binary search the prefix range, or build a trie for repeated queries. O(log n + m) per query after preprocessing.

HOW TO THINK ABOUT IT

If the dictionary is sorted, all words with a prefix form one contiguous block, and its edges are found with binary search: the first word >= prefix, and the first word that is >= prefix with its last character incremented. For many queries on a static list this beats a trie in memory.

Basis: Search Suggestions System (LeetCode 1268) and prefix queries on a sorted list.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, prefix):
    found = [w for w in words
             if w.startswith(prefix)]
    return found
```

BRUTE FORCE

TIME	SPACE
O(W * L) per query	O(m)

WHY IT IS SLOW

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION · Sort once, binary search the range

TIME	SPACE
O(log W + m) per query after O(W log W) sorting	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

FOLLOW-UP 2 Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

FOLLOW-UP 3 Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return dictionary words matching a prefix
#
# You are given a list of strings words (a dictionary) and
# a string prefix. Return all words in words that start
# with prefix. Clarifications/constraints: - Matching is
# case-sensitive. - If no words match, return an empty
# list. - If words contains duplicates, include duplicates
# in the output. - You may return the result in any order
# unless the interviewer asks for a specific ordering
# (e.g., lexicographic).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first K matches (autocomplete).
# 2. Words are added while queries run.
# 3. Case-insensitive matching.
#
# INTERVIEW NOTES
# If the dictionary is sorted, all words with a prefix
```

```
# form one contiguous block, and its edges are found with
# binary search: the first word >= prefix, and the first
# word that is >= prefix with its last character
# incremented. For many queries on a static list this
# beats a trie in memory.
#
# BRUTE FORCE (say it first): Filter the whole list per
# query
# Test startswith on every word. Correct and fine for one
# query; each query pays for the whole list. Time O(W * L)
# per query. Space O(m).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Filter the whole list per
# query | time O(W * L) per query
def prefix_matches_brute(words, prefix):
    return sorted(
        w for w in words if w.startswith(prefix)
    )


# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
```

```
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
# FOLLOW-UP 1: Only the first K matches (autocomplete).
# Answer: Stop the forward walk after K words, as the
# limit parameter does; with the sorted list the first K
# in alphabetical order are already the first K in the
# block.
# FOLLOW-UP 2: Words are added while queries run.
# Answer: Keep the list sorted with bisect.insort (O(n)
# per insert, fast in practice) or use a balanced tree; a
# trie handles inserts in O(L).
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)

    # FOLLOW-UP 1: limit= returns only the first K
    def matches(self, prefix, limit=None):
        # first possible position, then walk while it
        # matches
        lo = bisect_left(self.words, prefix)
        # one past the last word that starts with the
        # prefix
        hi = bisect_right(self.words, prefix + "")
        found = self.words[lo:hi]
        return found if limit is None else found[:limit]

    # FOLLOW-UP 2: words added while queries run
    def add(self, word):
        # words can arrive while queries run: keep it
        # sorted
        insort(self.words, word)


# SIMPLE SCAN (say it first)
def matches_scan(words, prefix):
    # the simple O(n * L) answer, in the original order
    return [w for w in words if w.startswith(prefix)]


# FOLLOW-UP 3: Case-insensitive matching.
# Answer: Store a casefolded sort key next to each word (a
# tuple (key, word)) and search on the key; do not
# lowercase the displayed word.
```

```
class CaseInsensitiveIndex:
    """Ignore case, but show the original words."""

    def __init__(self, words):
        keyed = ((w.casefold(), w) for w in words)
        self.items = sorted(keyed)

    def matches(self, prefix):
        key = prefix.casefold()
        lo = bisect_left(self.items, (key,))
        out = []
        while lo < len(self.items) and \
                self.items[lo][0].startswith(key):
            out.append(self.items[lo][1])
            lo += 1
        return out


if __name__ == "__main__":
    words = ["apple", "app", "apricot", "Apple", "banana",
             "app"]
    idx = PrefixIndex(words)
    # duplicates are kept; matching is case sensitive
    assert idx.matches("ap") == ["app", "app", "apple",
                                 "apricot"]
    assert idx.matches("x") == []
    assert idx.matches("") == sorted(words)
    assert idx.matches("ap", limit=2) == ["app", "app"]
    scan = sorted(matches_scan(words, "ap"))
    assert scan == idx.matches("ap")
    idx.add("apex")
    assert "apex" in idx.matches("ap")
    ci = CaseInsensitiveIndex(words)
    got = ci.matches("APP")
    assert got == ["app", "app", "Apple", "apple"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
(['apple', 'app', 'banana', 'apply'], 'app')	['apple', 'app', 'apply']
(['Cat', 'car', 'Cart', 'dog'], 'Ca')	['Cat', 'Cart']

STUDY HERE: topic 3, Trees and Tries

Site tags: data-structure, tree, medium

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 12 of 22

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 118/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

EXAMPLES

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

APPROACH

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

HOW TO THINK ABOUT IT

If k can change from query to query you need every value, in a structure that supports rank queries; if k is fixed you only need the k largest, and a min-heap of size k gives the answer at its root. Ask which of the two it is: it decides everything.

Basis: LeetCode 703, Kth Largest Element in a Stream, generalized so k is passed to each query (0-indexed in the examples).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

BRUTE FORCE

TIME	SPACE
O(n log n) per query	O(n)

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Examples: insert(3) insert(3) insert(2) findLargest(0) -
>
# 3 findLargest(1) -> 3 findLargest(2) -> 2
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. What if values can also be deleted?
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.
#
# BRUTE FORCE (say it first): Store everything and sort
# for every query
# Append values to a list; on a query sort it in
# descending order and read index k. Correct and the
# oracle; each query re-sorts everything. Time O(n log n)
# per query. Space O(n).


import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Store everything and sort
# for every query | time O(n log n) per query
class StreamKthBrute:
    def __init__(self):
        self.a = []

    def insert(self, x):
        self.a.append(x)

    def findLargest(self, k):
        s = sorted(self.a, reverse=True)
        return s[k] if 0 <= k < len(s) else None


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
```

```
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
# FOLLOW-UP 1: What if values can also be deleted?
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# FOLLOW-UP 3: Values can also be deleted.
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None


# FOLLOW-UP 2: What is the trade-off between the two
# designs?
# Answer: The heap is the smallest and fastest when k is
# fixed and small (O(log k), O(k) memory) but cannot
# change k or delete. The sorted list answers any k and is
# simplest, at O(n) memory and a linear shift per insert.
class FixedKth:
    """Fixed k, inserts only: a min-heap of the best k."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def insert(self, x):
        heapq.heappush(self.heap, x)
        # keep only the k largest values
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def kth_largest(self):
        # the smallest of the k largest
        if len(self.heap) < self.k:
            return None
        return self.heap[0]


if __name__ == "__main__":
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    top3 = [s.find_largest(k) for k in range(3)]
    assert top3 == [3, 3, 2]
    assert s.find_largest(3) is None
    # deletes work too
    assert s.delete(3) and not s.delete(9)
    assert [s.find_largest(k) for k in range(2)] == [3, 2]
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    # the two largest are 9 and 7, so the 2nd largest is 7
    assert f.kth_largest() == 7
```

NOTES FROM CANDIDATES

An order-statistics structure (size-augmented BST / balanced tree / skiplist) gives O(log n) for both ops.
An order-statistics Fenwick tree over a coordinate-compressed value range gives O(log V).
A naive sorted array gives O(log n) find but O(n) insert; mention as a baseline.
Two heaps (the canonical streaming-median pattern) only work for a fixed k; the interviewer wants arbitrary k per call.

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY HERE: topic 3, Trees and Tries

MEDIUM | PracHub | Technical Screen | 13 of 22

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

POSTED 2025-12-02 | 322 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 58 30 DAYS 0 PRIORITY 43/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Posted on PracHub: 2025-12-02 (updated 2026-05-02)

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:

QUESTION

- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

QUESTION

Tasks: 1.
Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.
Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.
Output:

- An integer `C` = minimal number of cars needed.
- A mapping from each `rental_id` to a `car_id` in the range `1..C`, such that no two rentals assigned to the same `car_id` overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals `[s1, e1)` and `[s2, e2)` overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if `e1 == s2` they do **not** over

APPROACH

Top-K messages via counts + heap; grid paths via DP (with obstacles if present); car rentals = minimum fleet = max overlapping reservations (sweep line).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three independent tasks. Chat: activity means the number of messages, ties resolved how? Grid paths: moves right or down only, with or without obstacles, modulo? Car rentals: minimum cars equals the maximum overlap of reservations; does a car returned at time t serve a rental starting at t?

BRUTE FORCE TO SAY OUT LOUD

Chat: sort the counts. Grid: recursion without memory. Cars: assign reservations to cars one by one.

OPTIMAL IDEA

Chat: Counter and heapq.nsmallest with the key (count descending, user id ascending). Grid: dp over cells with a rolling row, dp[j] += dp[j - 1], and 0 on obstacles; O(mn) time, O(n) space. Cars: sort the reservations by start and keep a min-heap of end times; reuse the earliest-ending car if it is free, else add one. The heap size at the end is the answer.

DRY-RUN THESE CASES

Chat: ties, fewer users than K. Grid: 1 x n, an obstacle on the start, no path. Cars: back-to-back rentals, one rental, all identical.

LIKELY FOLLOW-UPS

Chat in a stream, grid paths with diagonal moves, cars with a fixed fleet size and rejecting requests.

PITFALLS

Treating back-to-back as overlapping. Forgetting the modulo in the grid count if the statement requires it.

HOW TO THINK ABOUT IT

Three unrelated tasks, so say so and give each a one-line plan. Chat: count, then take the top K with a heap. Grid: each column depends only on the previous one, so keep one column. Cars: the minimum number of cars is the largest number of rentals that overlap at once, which a heap of return times tracks while also giving an assignment.

Basis: Three independent tasks: top K frequent items (LeetCode 347 / 692), grid path counting (a Unique Paths variant) and minimum meeting rooms with assignment (LeetCode 253 plus the assignment step).

BRUTE FORCE · Q1: sort all users by (count, user id)

BRUTE FORCE · PSEUDO-CODE

```
def top_k_users(log, k):
    counts = Counter(msg.user for msg in log)
    def rank(u):
        return (-counts[u], u)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE · Q2: plain recursion over the moves

BRUTE FORCE · PSEUDO-CODE

```
def ways(r, c):
    if c == n - 1:
        return 1 if r == n - 1 else 0
    total = 0
    for dr in (-1, 0, 1):
        if 0 <= r + dr < n:
            total += ways(r + dr, c + 1)
    return total
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

```
def cars_needed(rentals):
    free_at = []  # per car
    for start, end in sorted(rentals):
        for i, f in enumerate(free_at):
            if f <= start:
                free_at[i] = end  # reuse
                break
        else:
            free_at.append(end)  # new car
    return len(free_at)
```

TIME	SPACE
O(R^2)	O(R)

WHY IT IS SLOW

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION · Q1: top-K active users with a size-K heap

TIME	SPACE
O(n + d log k) for d distinct users	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: count constrained paths with a rolling column

TIME	SPACE
O(m * n)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q3: minimum cars and one valid assignment

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

FOLLOW-UP 2 Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

FOLLOW-UP 3 Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

FOLLOW-UP 4 Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve chat, grid paths, and car rentals
#
# You are given three independent programming tasks. For
# each, describe and implement an efficient algorithm, and
# be prepared to analyze its time and space complexity.
# --- ### Q1. Find top-K most active chat users You are
# given a log of chat messages from a messaging
# application. **Input**: - An array messages of length n,
# where each element messages[i] is the user_id (string or
# integer) of the sender of the i-th message. - An integer
# k (1 <= k <= number of distinct users). **Output**: -
# Return a list of k user IDs corresponding to the users
# who have sent the most messages. **Details &
# requirements**: - The "activity" of a user is defined as
# the number of messages they have sent (i.e., their
# frequency in messages). - If there are fewer than k
# distinct users, return all distinct users.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1: what if the log is a stream and memory is
# limited?
# 2. Q2: what changes if there are obstacles or the path
# may also go straight up and down?
# 3. Q3: how do you answer 'is one more rental possible'
# online?
# 4. Q2 with obstacles.
#
# INTERVIEW NOTES
# Three unrelated tasks, so say so and give each a one-
# line plan. Chat: count, then take the top K with a heap.
# Grid: each column depends only on the previous one, so
# keep one column. Cars: the minimum number of cars is the
# largest number of rentals that overlap at once, which a
# heap of return times tracks while also giving an
# assignment.
#
```

```
# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id)
# Count the messages and sort all distinct users, then
# take the first K. Correct and the oracle; sorting all d
# users does more work than a heap of size K. Time O(d log
# d). Space O(d).


import heapq
```

```
from collections import Counter


# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id) | time O(d log d)
def top_users_sort(messages, k):
    c = Counter(messages)
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]


# Q1 second approach: sort every user
def top_k_users_sorted(messages, k):
    # Q1, second approach: sort every user, O(d log d)
    counts = Counter(messages)
    ranked = sorted(counts, key=lambda u: (-counts[u], u))
    return ranked[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
```

```
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            # FOLLOW-UP 4: blocked cells (obstacles) give
            # no paths
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]
```

```
# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)


# FOLLOW-UP 3: Q3: how do you answer 'is one more rental
# possible' online?
# Answer: Keep the busy heap and the free-car count
# between calls. A new rental fits without a new car if
# any car's return time is <= its pickup, which is a heap
# peek after popping everything that returned before the
```

```
# pickup.
class RentalDesk:
    """Online: can one more rental be served right now?"""

    def __init__(self, fleet):
        self.fleet, self.out = fleet, []

    def rent(self, pickup, back):
        # cars returned by `pickup` are free again
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True


if __name__ == "__main__":
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    # grid paths: a 3 x 3 grid has 2 paths along the
    # bottom
    assert count_paths(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    # an obstacle in the middle of the bottom row
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1
    modulus = 10 ** 9 + 7
    assert count_paths(5, 30, mod=modulus) < modulus
    cars = [(1, 4), (2, 5), (4, 6), (5, 8)]
    assert min_cars(cars) == 2
    assert min_cars([]) == 0
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

FULL QUESTION TEXT FROM THE SITE

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

STUDY HERE: topic 3, Trees and Tries

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 14 of 22

Find mode in a trinary search tree

POSTED 2025-11-16 | 338 DAYS AGO | OLDER

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 137/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Posted on PracHub: 2025-11-16 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val N
ode middle; // all values = val Node right; // all val
ues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

QUESTION

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []

QUESTION

Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

APPROACH

In-order traversal (left, middle, right) so equal values are consecutive; track current run length and the max; O(h) extra space.

HOW TO THINK ABOUT IT

In a valid search tree an in-order walk (left, middle, right) visits values in sorted order, so equal values are adjacent and you can count runs with a single running counter and O(height) memory instead of a dictionary. If the tree may be broken, the order guarantee is gone and you need a real frequency table.

Basis: LeetCode 501, Find Mode in Binary Search Tree, with a middle child for equal values. Valid case: left < val, middle = val, right > val.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def modes(root):
    counts = Counter()
    stack = [root]
    while stack:
        node = stack.pop()
        if node:
            counts[node.val] += 1
            stack += [node.left, node.middle,
                      node.right]
    top = max(counts.values())
    return [v for v in counts
            if counts[v] == top]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION · In-order traversal counting runs (O(height) extra space)

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

FOLLOW-UP 2 Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

FOLLOW-UP 3 The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. **Cost:** O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find mode in a trinary search tree
#
# You are given the root of a **trinary search tree**
# where each node has up to three children: text class
# Node { int val; Node left; // all values < val Node
# middle; // all values = val Node right; // all values >
# val } Assume the tree is intended to satisfy the
# ordering rules above. ## Task Return the **mode(s)** of
# the tree: the value(s) that occur most frequently among
# all nodes. - If there is one mode, return a single-
# element list. - If multiple values tie for highest
# frequency, return **all** such values (in any order). ##
# Constraints (typical) - Number of nodes n can be large
# (e.g., up to 10^5). - Values can be any 32-bit signed
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree may be broken (it does not respect the
# ordering).
# 2. Only O(1) extra space (no stack, no dictionary).
# 3. The tree is huge and values arrive in a stream.
#
# INTERVIEW NOTES
# In a valid search tree an in-order walk (left, middle,
# right) visits values in sorted order, so equal values
```

```
# are adjacent and you can count runs with a single
# running counter and O(height) memory instead of a
# dictionary. If the tree may be broken, the order
# guarantee is gone and you need a real frequency table.
#
# BRUTE FORCE (say it first): Collect every value in a
# Counter
# Traverse the whole tree, count each value and return the
# ones with the highest count. Correct for any tree (even
# a broken one) and the oracle; it uses memory
# proportional to the number of distinct values. Time
# O(n). Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Collect every value in a
# Counter | time O(n)
def modes_counter(root):
```

```
    c, stack = Counter(), [root] if root else []
    while stack:
        x = stack.pop()
        c[x.val] += 1
        stack.extend(
            ch
            for ch in (x.left, x.middle, x.right)
            if ch
        )
    if not c:
        return []
    top = max(c.values())
    return sorted(v for v, k in c.items() if k == top)


# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
```

```
    return sorted(modes)


# FOLLOW-UP 1: The tree may be broken (it does not respect
# the ordering).
# Answer: In-order no longer gives sorted values, so equal
# values need not be adjacent. Fall back to a frequency
# table over a full traversal (the Counter version): O(n)
# time, O(distinct values) space.
def find_modes_counter(root):
    # a broken tree (ordering not respected): count
    # everything
    counts, stack = Counter(), [root] if root else []
    while stack:
        node = stack.pop()
        counts[node.val] += 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    if not counts:
        return []
    top = max(counts.values())
    return sorted(v for v in counts if counts[v] == top)


# HELPERS for the checks below
class TNode:
    def __init__(self, val, left=None, middle=None,
                 right=None):
        # left < val, middle == val, right > val
        self.val, self.left = val, left
        self.middle, self.right = middle, right


if __name__ == "__main__":
    #        5
    #     /  |  \
    #    3   5   8
    #   /|       |
    #  3 3       8
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    tree = TNode(5, threes, TNode(5),
                 TNode(8, None, TNode(8)))
    assert find_modes(tree) == [3]
    assert find_modes_counter(tree) == [3]
```

```
    # a tie between 1 and 2
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes_counter(tie) == [2]
    assert find_modes(None) == []
    assert find_modes_counter(None) == []
    tie2 = TNode(2, TNode(1), None, TNode(3))
    assert find_modes(tie2) == [1, 2, 3]
```

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: l
ist[int], right: list[int], root: int) -> list[int]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

STUDY HERE: topic 3, Trees and Tries

MEDIUM | PracHub | Technical Screen | 15 of 22

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

POSTED 2025-08-04 | 442 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 29/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Posted on PracHub: 2025-08-04 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters. https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/ https://leetcode.com/problems/course-schedule/description/ https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

QUESTION

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

APPROACH

BFS by level with reversed alternate levels; topological sort / DFS cycle detection; plus a sliding-window task.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three LeetCode problems: 103 zigzag level order, 207 course schedule, 340 longest substring with at most K distinct characters. For each ask the usual edge questions: empty tree, duplicate prerequisites, K = 0.

BRUTE FORCE TO SAY OUT LOUD

Zigzag: do a normal level order and reverse alternate levels afterwards. Course schedule: DFS from every course to find cycles. Substring: check every substring.

OPTIMAL IDEA

Zigzag: BFS level by level, reverse (or use a deque appended on the left or right) on every other level. O(n). Course schedule: Kahn's algorithm, with in-degree counts and a queue; if processed courses equal the course count there is no cycle. O(V + E). Substring (340): sliding window with a dict of counts; expand right, and while distinct characters exceed K shrink from the left, deleting a character at count 0. Track the maximum window. O(n).

DRY-RUN THESE CASES

Zigzag: empty tree, a single node, a skewed tree. Course schedule: no prerequisites, a self loop, a two-course cycle. Substring: K = 0 (answer 0), K larger than distinct characters, a repeated character.

LIKELY FOLLOW-UPS

Course Schedule II (return an order), longest substring with exactly K distinct, at most K distinct in a stream.

PITFALLS

Reversing the wrong levels. Not removing a key from the window dict when its count hits zero.

HOW TO THINK ABOUT IT

Three standard patterns. Level order needs a queue and a direction flag per level. 'Can everything be finished' is cycle detection in a directed graph: Kahn's algorithm removes nodes with no remaining prerequisites, and a cycle leaves some behind. 'Longest window with at most K of something' is a sliding window with a count map that shrinks from the left when the limit is exceeded.

Basis: Three LeetCode problems: 103 Binary Tree Zigzag Level Order, 207 Course Schedule and 340 Longest Substring with At Most K Distinct Characters.

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE · Longest window: check every substring

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, K):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) > K:
                break
            best = max(best, j - i + 1)
    return best
```

TIME	SPACE
O(n^2)	O(k)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE · Zigzag: level order, then reverse the odd levels

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION · Course schedule: Kahn's topological sort

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Longest window: sliding window with a count map

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Zigzag: fill each level from the correct end

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

FOLLOW-UP 2 Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

FOLLOW-UP 3 Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve tree, graph, sliding-window
# problems
#
# ##### Question LeetCode 103. Binary Tree Zigzag Level
# Order Traversal – given a binary tree root, return its
# zigzag level-order traversal. LeetCode 207. Course
# Schedule – given course prerequisites, determine if all
# courses can be finished by detecting cycles with
# topological sort. LeetCode 340. Longest Substring with
# At Most K Distinct Characters – given string s and
# integer k, return the length of the longest substring
# containing at most k distinct characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Course Schedule II: return an order in which to take
# the courses.
# 2. Longest substring with exactly K distinct characters.
# 3. Zigzag on an n-ary tree, or the right-to-left
# variant.
#
# INTERVIEW NOTES
# Three standard patterns. Level order needs a queue and a
# direction flag per level. 'Can everything be finished'
# is cycle detection in a directed graph: Kahn's algorithm
# removes nodes with no remaining prerequisites, and a
# cycle leaves some behind. 'Longest window with at most K
# of something' is a sliding window with a count map that
# shrinks from the left when the limit is exceeded.
#
# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle
# From each course follow prerequisites; if the walk
# returns to the start there is a cycle. Correct and the
# oracle; the same parts of the graph are walked again and
# again. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle | time O(V * (V + E))
def can_finish_dfs(n, prereqs):
    adj = defaultdict(list)
    for a, b in prereqs:
        adj[b].append(a)

    def reaches(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches(c) for c in range(n))


# HELPERS for the base question
# FOLLOW-UP 1: Course Schedule II: return an order in
# which to take the courses.
# Answer: The same Kahn loop records the order in which
# courses are removed; if fewer than n courses are
# removed, return an empty list.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
```

```
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []


# HELPERS for follow-up 2
```

```
def count_at_most_k(s, k):
    # number of substrings with at most k distinct letters
    if k < 1:
        return 0
    count, left, total = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1
    return total


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out


# BASE QUESTION (optimal): Course schedule: Kahn's
```

```
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
```

```
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# FOLLOW-UP 2: Longest substring with exactly K distinct
# characters.
# Answer: Take atMost(K) minus atMost(K - 1) for counting
# substrings, or keep the window and only record lengths
# when the distinct count is exactly K (careful with the
# shrink).
def count_exactly_k(s, k):
    # exactly k = at most k minus at most k - 1
    most = count_at_most_k(s, k)
    return most - count_at_most_k(s, k - 1)


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val, self.left, self.right = val, left, right


def build(values):
    # level-order list, None for a missing child
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


if __name__ == "__main__":
    tree = build([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    order = course_order(4, [(1, 0), (2, 1), (3, 2)])
    assert order == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
([3, 9, 20, None, None, 15, 7],)	[[3], [20, 9], [15, 7]]
([1],)	[[1]]

STUDY HERE: topic 3, Trees and Tries

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 16 of 22

Implement zigzag level-order traversal

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TALK ALL TIME 38.7 30 DAYS 0 PRIORITY 96/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

APPROACH

BFS by level; reverse (or use a deque appending left/right) on alternate levels. O(n).

HOW TO THINK ABOUT IT

It is a plain level-order traversal where every second level is read in the other direction. Either collect each level and reverse the odd ones, or fill each level from the correct end of a deque so no reversal is needed.

Basis: LeetCode 103, Binary Tree Zigzag Level Order Traversal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION · BFS filling each level from the left or the right

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

FOLLOW-UP 2 Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

FOLLOW-UP 3 N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement zigzag level-order traversal
#
# Given the root of a binary tree, return the node values
# in zigzag level order: visit nodes level by level,
# reading the first level from left to right, the next
# from right to left, and so on alternating each level.
# Output a list of lists of integers where each inner list
# contains the values for one level in the required
# direction. Aim for O(n) time and O(w) extra space, where
# n is the number of nodes and w is the maximum width of
# the tree.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up level order (leaf level first).
# 2. Do it with DFS.
# 3. N-ary tree.
#
# INTERVIEW NOTES
# It is a plain level-order traversal where every second
# level is read in the other direction. Either collect
# each level and reverse the odd ones, or fill each level
# from the correct end of a deque so no reversal is
# needed.
#
# BRUTE FORCE (say it first): Level order, then reverse
# every second level
```

```python
# Do a normal breadth-first traversal collecting each
# level, and reverse the levels with odd index at the end.
# Easy to get right, with one extra pass over those
# levels. Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Level order, then reverse
# every second level | time O(n)
def zigzag_reverse(root):
    levels, queue = [], deque([root] if root else [])
    while queue:
        levels.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(
                c for c in (n.left, n.right) if c
            )
    return [
        lv[::-1] if i % 2 else lv
        for i, lv in enumerate(levels)
    ]


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out


# FOLLOW-UP 1: Bottom-up level order (leaf level first).
# Answer: Run the normal traversal and reverse the list of
# levels at the end.
def level_order_bottom_up(root):
    # leaf level first: the normal traversal, reversed at
    # the end
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out[::-1]


# FOLLOW-UP 2: Do it with DFS.
# Answer: Pass the depth; append the value to
# levels[depth], at the end when the depth is even and at
# the front (insert at 0 or a deque) when odd. Recursion
# depth is the tree height.
def zigzag_dfs(root):
    # DFS: pass the depth; odd levels are filled at the
    # front
    levels = []

    def go(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append(deque())
        if depth % 2 == 0:
            levels[depth].append(node.val)
        else:
            levels[depth].appendleft(node.val)
        go(node.left, depth + 1)
        go(node.right, depth + 1)

    go(root, 0)
    return [list(x) for x in levels]


# FOLLOW-UP 3: N-ary tree.
# Answer: Replace the two children by the list of
# children; the direction flag works the same way.
def zigzag_nary(root):
    # an n-ary tree: a list of children instead of two
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        vals = [n.val for n in queue]
        out.append(vals if ltr else vals[::-1])
        for _ in range(len(queue)):
            queue.extend(queue.popleft().children)
        ltr = not ltr
    return out



# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


class NNode:
    def __init__(self, val, children=()):
        self.val, self.children = val, list(children)


if __name__ == "__main__":
    right = TreeNode(20, TreeNode(15), TreeNode(7))
    tree = TreeNode(3, TreeNode(9), right)
    want = [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(tree) == want
    assert zigzag_dfs(tree) == want
    up = level_order_bottom_up(tree)
    assert up == [[15, 7], [9, 20], [3]]
    assert zigzag_level_order(None) == []
    assert zigzag_dfs(None) == []
    kids = [NNode(2, [NNode(5)]), NNode(3), NNode(4)]
    root = NNode(1, kids)
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]
```

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[list[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY HERE: topic 3, Trees and Tries

MEDIUM | IN APP LC 1650 | HackTheRounds | Phone Screen | 17 of 22

Lowest Common Ancestor of a Binary Tree III

STUDY: TALK ALL TIME 14.7 30 DAYS 0 PRIORITY 163/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1650 Lowest Common Ancestor of a Binary Tree III

LeetCode match: LC 1650

QUESTION

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA). Each node has a parent pointer.

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`The LCA of nodes 5 and 1 is 3.`

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`The LCA of nodes 5 and 4 is 5.`

Example 3	
Input	`root = [1,2], p = 1, q = 2`
Output	`1`
Why	`Node 1 is ancestor of node 2.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Parent pointers: two-pointer swap trick, or a set of ancestors

HOW TO THINK ABOUT IT

With parent pointers each node is the head of a linked list that ends at the root, so the question is where two linked lists meet. If the lists have different lengths, walk each pointer to the end and then restart it on the other node: both pointers then travel the same total distance and arrive at the meeting node together.

Basis: LeetCode 1650, Lowest Common Ancestor of a Binary Tree III (every node has a parent pointer).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lca(p, q):
    ancestors = set()
    node = p
    while node:
        ancestors.add(node)
        node = node.parent
    node = q
    while node:
        if node in ancestors:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Uses O(h) extra space for the set. Two pointers that switch to the other node's start when they fall off the root meet at the answer with O(1) space.

OPTIMAL SOLUTION · Two pointers that switch to the other node at the end

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

FOLLOW-UP 2 Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

FOLLOW-UP 3 The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Lowest Common Ancestor of a Binary Tree
# III
#
# Given two nodes of a binary tree p and q, return their
# lowest common ancestor (LCA). Each node has a parent
# pointer.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without the swap trick (it feels like magic).
# 2. Many LCA queries on the same tree.
# 3. The nodes might not be in the same tree.
#
# INTERVIEW NOTES
# With parent pointers each node is the head of a linked
# list that ends at the root, so the question is where two
# linked lists meet. If the lists have different lengths,
# walk each pointer to the end and then restart it on the
# other node: both pointers then travel the same total
# distance and arrive at the meeting node together.
#
# BRUTE FORCE (say it first): Collect the ancestors of one
# node in a set
# Walk from p to the root storing every ancestor
# (including p), then walk from q upward until you hit a
# stored node. Simple and correct; it uses memory for the
# ancestor set. Time O(h). Space O(h).


def brute(p, q):
    # oracle: the first ancestor of q that is also one of
    # p
    above = set()
    while p:
        above.add(p)
        p = p.parent
    while q and q not in above:
        q = q.parent
    return q


# BRUTE FORCE (say it first): Collect the ancestors of one
# node in a set | time O(h)
class PNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.parent = (
            val,
            None,
            None,
            None,
        )


# HELPERS for the base question
def lca_set(p, q):
    seen = set()
    while p:
        seen.add(id(p))
        p = p.parent
    while q:
        if id(q) in seen:
            return q
        q = q.parent
    return None


# HELPERS for follow-up 1
def depth(node):
    d = 0
    while node.parent:
        node = node.parent
        d += 1
    return d


# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node.
def lca_swap(p, q):
    # two pointers climb to the root; at the top each one
    # jumps to the other node's start. Both walk the same
    # total distance, so they meet at the LCA (or both
    # reach
    # None together when the nodes are in different
    # trees).
    a, b = p, q
    while a is not b:
        a = a.parent if a else q
        b = b.parent if b else p
    return a


# FOLLOW-UP 1: Do it without the swap trick (it feels like
# magic).
# Answer: Compute the depth of each node by walking up,
# lift the deeper one by the difference, then step both up
# together until they meet. Same O(h) time and O(1) space
# and easier to justify.
def lca_by_depth(p, q):
    # follow-up: no trick. Lift the deeper node by the
    # difference in depth, then climb both until they
    # meet.
    dp, dq = depth(p), depth(q)
    while dp > dq:
        p, dp = p.parent, dp - 1
    while dq > dp:
        q, dq = q.parent, dq - 1
    while p is not q:
        p, q = p.parent, q.parent
        if p is None or q is None:
            return None
    return p


# FOLLOW-UP 2: Many LCA queries on the same tree.
# Answer: Preprocess with binary lifting (up[k][v] is the
# 2^k-th ancestor): O(n log n) to build, O(log n) per
# query. Or an Euler tour with a range-minimum structure
# for O(1) queries.
class LiftingLCA:
    """Follow-up: many queries on one tree.
    up[k][i] is the 2**k-th ancestor of node i: O(n log n)
    to build and O(log n) per query."""

    def __init__(self, root):
        self.ids = {root: 0}
        self.nodes = [root]
        parent, self.depth = [0], [0]
```

```
        # the list grows while we walk it: a BFS order
        for node in self.nodes:
            at = self.ids[node]
            for child in (node.left, node.right):
                if child:
                    self.ids[child] = len(self.nodes)
```

```
                    self.nodes.append(child)
                    parent.append(at)
                    self.depth.append(self.depth[at] + 1)
        self.up = [parent]
        count = len(self.nodes)
        while (1 << len(self.up)) < count:
            prev = self.up[-1]
            row = [prev[prev[i]] for i in range(count)]
            self.up.append(row)

    def query(self, p, q):
        a, b = self.ids[p], self.ids[q]
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        # lift the deeper node to the same depth
        diff = self.depth[a] - self.depth[b]
        for k, row in enumerate(self.up):
            if diff >> k & 1:
                a = row[a]
        if a == b:
            return self.nodes[a]
        # lift both as high as possible while they differ
        for row in reversed(self.up):
            if row[a] != row[b]:
                a, b = row[a], row[b]
        return self.nodes[self.up[0][a]]


# HELPERS for the checks below
class Node:
    def __init__(self, val, parent=None):
        self.val, self.parent = val, parent
        self.left = self.right = None


def attach(parent, val, side):
    # create a child and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def random_tree(size):
    import random
    root = Node(0)
```

```
    nodes = [root]
    while len(nodes) < size:
        parent = random.choice(nodes)
        side = random.choice(["left", "right"])
        if getattr(parent, side) is None:
            nodes.append(attach(parent, len(nodes), side))
    return nodes


if __name__ == "__main__":
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    assert lca_swap(six, four) is five
    assert lca_swap(five, four) is five
    assert lca_swap(six, one) is root
    assert lca_swap(root, root) is root
```

```
    other = Node(99)
    assert lca_swap(six, other) is None
    assert lca_by_depth(six, other) is None
    assert lca_by_depth(four, one) is root
    index = LiftingLCA(root)
    assert index.query(six, four) is five
    assert index.query(root, four) is root
    import random
    for _ in range(100):
        nodes = random_tree(random.randint(1, 12))
        index = LiftingLCA(nodes[0])
        for _ in range(20):
            p = random.choice(nodes)
            q = random.choice(nodes)
            want = brute(p, q)
            assert lca_swap(p, q) is want
            assert lca_by_depth(p, q) is want
            assert index.query(p, q) is want
```

STUDY HERE: topic 3, Trees and Tries

MEDIUM | IN APP LC 236 | HackTheRounds | Onsite | 18 of 22

Lowest Common Ancestor

STUDY: TALK | ALL TIME 41.5 | 30 DAYS 0 | PRIORITY 88/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #236 Lowest Common Ancestor of a Binary Tree

LeetCode match: LC 236

QUESTION

Given a binary tree and two nodes p and q, find their lowest common ancestor (LCA): the lowest node that has both p and q as descendants (a node can be a descendant of itself).
Follow-up: what if it is an n-ary tree?

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`LCA of 5 and 1 is 3.`

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`A node can be its own ancestor.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Recursive DFS; follow-up: n-ary tree

HOW TO THINK ABOUT IT

Ask each subtree 'do you contain p or q?'. The first node where both subtrees report a hit (or the node is p or q itself and a subtree has the other) is the answer. One pass returns the answer upward: a node returns itself if p or q, the child result if only one side found something, and itself if both sides did.

Basis: LeetCode 236, Lowest Common Ancestor of a Binary Tree (and its n-ary follow-up).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def path_to(root, target):
    if root is None:
        return None
    if root is target:
        return [root]
    for child in root.children:
        sub = path_to(child, target)
        if sub:
            return [root] + sub
    return None
def lca(root, p, q):
    path_p = path_to(root, p)
    path_q = path_to(root, q)
    answer = None
    for a, b in zip(path_p, path_q):
        if a is not b:
            break
        answer = a
    return answer
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Two full paths stored, O(n) space, and two traversals. One recursive pass that reports whether each subtree contains p or q finds the answer in O(n) time and O(h) space.

OPTIMAL SOLUTION · One recursive pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

FOLLOW-UP 2 A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(n) time and space, no recursion.

FOLLOW-UP 3 p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Lowest Common Ancestor
#
# Given a binary tree and two nodes p and q, find their
# lowest common ancestor (LCA): the lowest node that has
# both p and q as descendants (a node can be a descendant
# of itself). Follow-up: what if it is an n-ary tree?
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is n-ary.
# 2. A very deep tree, or no recursion allowed.
# 3. p or q might not be in the tree.
#
# INTERVIEW NOTES
# Ask each subtree 'do you contain p or q?'. The first
# node where both subtrees report a hit (or the node is p
# or q itself and a subtree has the other) is the answer.
# One pass returns the answer upward: a node returns
# itself if p or q, the child result if only one side
# found something, and itself if both sides did.
#
# BRUTE FORCE (say it first): Find the root-to-node path
# for each node and compare
# Compute the path from the root to p and to q, then the
# last common node on the two paths is the LCA. Correct
# and easy to explain; it stores two full paths. Time
# O(n). Space O(n).


def brute(root, p, q):
    # oracle: the deepest node whose subtree holds both
    def holds(node, target):
        return node is target or any(
            holds(c, target) for c in node.children)

    best, stack = None, [root]
    while stack:
        node = stack.pop()
        if holds(node, p) and holds(node, q):
            best = node
            stack.extend(node.children)
    return best


# BRUTE FORCE (say it first): Find the root-to-node path
# for each node and compare | time O(n)
```

```
def _path(root, target):
    stack = [(root, [root])]
    while stack:
        n, path = stack.pop()
        if n is target:
            return path
        for c in (n.left, n.right):
            if c:
                stack.append((c, path + [c]))
    return []


# HELPERS for the base question
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = (
            val,
            left,
            right,
        )
```

```
def lca_paths(root, p, q):
    a, b = _path(root, p), _path(root, q)
    lca = None
    for x, y in zip(a, b):
        if x is y:
            lca = x
        else:
            break
    return lca


# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
# up.
# FOLLOW-UP 1: The tree is n-ary.
# Answer: The same recursion: collect the non-empty
# results from all children; if two or more children
# reported a hit this node is the answer, otherwise pass
# the single result (or None) up.
def lca(root, p, q):
    # each call reports where p or q was found below. A
```

```
    # node
    # with hits from two different children (or that is p
    # or
    # q itself) is the answer; a single hit is passed up.
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        # FOLLOW-UP 1: n-ary: collect hits from every
        # child
        hits = [r for r in map(find, node.children) if r]
        if len(hits) >= 2:
            return node
        return hits[0] if hits else None

    return find(root)


# FOLLOW-UP 2: A very deep tree, or no recursion allowed.
# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def parents_of(root):
    # follow-up: one traversal with a stack (no recursion)
    # that records every node's parent
    parent, stack = {root: None}, [root]
    while stack:
        node = stack.pop()
        for child in node.children:
            parent[child] = node
            stack.append(child)
    return parent


# FOLLOW-UP 2: A very deep tree, or no recursion allowed.
# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def lca_iterative(root, p, q):
    # climb from p collecting its ancestors, then climb
    # from
    # q until we meet one of them: O(n) time and space
```

```
    parent = parents_of(root)
    above = set()
    node = p
    while node:
        above.add(node)
        node = parent[node]
    node = q
    while node not in above:
        node = parent[node]
    return node


# FOLLOW-UP 3: p or q might not be in the tree.
# Answer: The recursive answer is wrong in that case (it
# returns the one that exists). Count how many of p and q
# were seen during the DFS, or first check both exist, and
# return None if not.
def lca_checked(root, p, q):
    # follow-up: p or q may be missing from the tree. The
    # plain recursion would return the one that exists, so
    # first check that both are really in the tree.
    parent = parents_of(root)
    if p not in parent or q not in parent:
        return None
    return lca_iterative(root, p, q)


# HELPERS for the checks below
class Node:
    def __init__(self, val, children=None):
        self.val = val
        # two children for a binary tree, any number for
        # n-ary
        self.children = children or []


if __name__ == "__main__":
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    root = Node(3, [five, one])
    assert lca(root, six, four) is five
    assert lca(root, five, four) is five
    assert lca(root, six, one) is root
```

```
    # an n-ary tree: three children under one node
    kids = [Node(i) for i in range(3)]
    top = Node("top", [Node("mid", kids), Node("side")])
    assert lca(top, kids[0], kids[2]).val == "mid"
    found = lca_iterative(top, kids[0], kids[2])
    assert found.val == "mid"
    stranger = Node(404)
    assert lca_checked(root, six, stranger) is None
    assert lca_checked(root, six, four) is five
    # a very deep tree: the iterative version has no limit
    chain = Node(0)
    cur = chain
    for i in range(1, 5000):
        child = Node(i)
        cur.children.append(child)
        cur = child
    assert lca_iterative(chain, cur, chain) is chain
    import random
    for _ in range(100):
        # a random tree: every new node gets a random
        # parent
        nodes = [Node(0)]
        for i in range(1, random.randint(1, 10)):
            child = Node(i)
            random.choice(nodes).children.append(child)
            nodes.append(child)
        p, q = random.choice(nodes), random.choice(nodes)
        want = brute(nodes[0], p, q)
        assert lca(nodes[0], p, q) is want
        assert lca_iterative(nodes[0], p, q) is want
```

STUDY HERE: topic 3, Trees and Tries

HARD | IN APP LC 1325 | PracHub | Technical Screen | 19 of 22

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 10.6 30 DAYS 0 PRIORITY 164/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

QUESTION

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

APPROACH

Post-order DFS: process children first, then delete the node if it became a leaf with value == target. O(n).

HOW TO THINK ABOUT IT

'Repeatedly' hides an order: a node becomes deletable only after its children are processed. That is a post-order dependency, so a single bottom-up pass does what many passes would do. Whenever a rule says 'repeat until nothing changes' on a tree, ask whether children-first order collapses it to one pass.

Basis: LeetCode 1325, Delete Leaves With a Given Value.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION · One post-order pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

FOLLOW-UP 3 Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.
#
# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing
# Each pass deletes only the current leaves with the
# target value; a parent that becomes a matching leaf
# waits for the next pass. On a chain of n matching nodes
# that is n passes of O(n) each. Time O(n * h). Space
# O(h).


from collections import deque


# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing | time O(n * h)
def remove_leaves_brute(root, target):
    def one_pass(node):
        if node is None:
            return None, False
        if node.left is None and node.right is None:
            return (
                (None, True)
                if node.val == target
                else (node, False)
            )
        node.left, a = one_pass(node.left)
        node.right, b = one_pass(node.right)
        return node, a or b

    changed = True
    while changed:
        root, changed = one_pass(root)
    return root


# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted


# FOLLOW-UP 1: The tree is a 100,000-node chain. How do
# you avoid a stack overflow?
# Answer: Collect nodes in pre-order with an explicit
# stack and remember each node's parent, then walk that
# list in reverse (children before parents) and unlink
```

```
# matching leaves. This keeps the post-order guarantee
# without recursion.
def remove_leaves_iterative(root, target):
    # no recursion: safe for a 100,000-node chain
    if root is None:
        return None, 0
    parent, order, stack = {root: None}, [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    # reversed order: children are handled before parents
    for node in reversed(order):
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
```

```
    # level-order list with None for missing children
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        # next two values are this node's children
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


def to_list(root):
    # the same encoding, with trailing Nones removed
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        out.append(node.val if node else None)
        if node:
            queue.append(node.left)
            queue.append(node.right)
    while out and out[-1] is None:
        out.pop()
    return out


def remove_target_leaves(values, target):
    root, _ = remove_leaves(build(values), target)
    return to_list(root)
```

```
if __name__ == "__main__":
    tree = [1, 2, 3, 2, None, 2, 4]
    result = remove_target_leaves(tree, 2)
    assert result == [1, None, 3, None, 4]
    # the whole tree can disappear
    assert remove_target_leaves([1, 1, 1], 1) == []
    assert remove_target_leaves([], 5) == []
    root, count = remove_leaves(build(tree), 2)
    assert count == 3
```

```
    again = remove_leaves_iterative(build(tree), 2)
    assert again[1] == 3
    assert to_list(again[0]) == to_list(root)
    # a 100,000 node chain works without recursion
    chain = TreeNode(7)
    cur = chain
    for _ in range(99999):
        cur.left = TreeNode(7)
        cur = cur.left
    gone = remove_leaves_iterative(chain, 7)
    assert gone == (None, 100000)
```

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
`([5, 7, 7, 7, 3, None, 7], 7)`	`[5, 7, None, None, 3]`
`([3, 3, 1], 3)`	`[3, None, 1]`

STUDY HERE: topic 3, Trees and Tries

Site tags: tree, heap, greedy, hard

HARD | 1Point3Acres | onsite-coding | 20 of 22

Huffman Tree Construction

ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 189/228**

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

EXAMPLES

```
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}
    -> standard Huffman tree with f near the root.
```

APPROACH

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to return the tree, a code table, or the encoded message, how ties are broken, and what to do with one symbol.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly scan for the two smallest weights. O(n^2).

OPTIMAL IDEA

Min-heap of nodes keyed by (weight, counter). Pop two, merge under a new node with the summed weight, push back, until one node remains. Walk the tree assigning '0' to left and '1' to right. O(n log n). Decode by walking the tree bit by bit. The counter prevents comparing nodes on ties.

DRY-RUN THESE CASES

The CLRS example (total cost 224, 'f' gets a one-bit code). One symbol (code '0'). Equal weights.

LIKELY FOLLOW-UPS

Canonical codes, decoding, or the two-queue O(n) method on sorted input.

PITFALLS

Comparing tree nodes on equal weights. Forgetting the single symbol case.

HOW TO THINK ABOUT IT

The two rarest symbols belong at the deepest level as siblings, so merge them first, treat the merged node as a new symbol with the combined weight, and repeat. 'Take the two smallest' is the job of a min-heap; tie-breaking with a counter keeps the heap from comparing tree nodes.

Basis: Classic greedy algorithm (CLRS 16.3); related to LeetCode 1167, Minimum Cost to Connect Sticks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Huffman Tree Construction
#
# Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16,
# "f": 45} -> standard Huffman tree with f near the root.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.
#
# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge
# Keep a list of (weight, tree) items; each round scan for
# the two smallest, merge, and put the result back. The
# same greedy as the heap version, with a linear scan
# instead of a heap operation. Returns the total cost (sum
# of weight * code length). Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge | time O(n^2)
def huffman_cost_scan(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]
        cost += a + b
        items = items[2:] + [a + b]
    return cost
```

```
# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# FOLLOW-UP 2: How do you make the code canonical, and why
# would you?
# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# FOLLOW-UP 4: Make the codes canonical so only the code
# lengths need to be stored.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
```

```
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# FOLLOW-UP 3: Time complexity if symbols arrive already
# sorted?
# Answer: Two-queue method: one queue holds the sorted
# leaves, the other the merged nodes (which are produced
# in nondecreasing order). Always pop the smaller front of
# the two queues. That is O(n) after sorting.
def lengths_two_queues(sorted_items):
    # input already sorted by weight: O(n) with two queues
```

```
    from collections import deque
    leaves = deque((w, [s]) for s, w in sorted_items)
    merged = deque()
    depth = {s: 0 for s, _ in sorted_items}

    def pop_smaller():
        # the smaller front of the two queues
        if not merged or (leaves and
                          leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        w1, a = pop_smaller()
        w2, b = pop_smaller()
        for s in a + b:
            depth[s] += 1
        merged.append((w1 + w2, a + b))
    return depth


# HELPERS for the checks below
def decode(bits, codes):
    # read bits until they match a code (prefix-free)
    back = {v: k for k, v in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in back:
            out.append(back[cur])
            cur = ""
    return "".join(out)


if __name__ == "__main__":
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    lengths = huffman_lengths(freq)
    # the most frequent symbol is next to the root
    assert lengths["f"] == 1
    # total cost of the optimal tree
    assert sum(freq[s] * lengths[s] for s in freq) == 224
    codes = huffman_codes(freq)
    # prefix-free: no code starts another code
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)
    text = "abfadc"
    bits = "".join(codes[ch] for ch in text)
    assert decode(bits, codes) == text
    items = sorted(freq.items(), key=lambda x: x[1])
    two = lengths_two_queues(items)
    assert sum(freq[s] * two[s] for s in freq) == 224
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}
```

NOTES FROM CANDIDATES

L6 onsite — interviewer expects clean code with a documented tree node, plus a discussion of encoding / decoding correctness.
Common follow-up: how to produce the code table (DFS from root, accumulating bits).
Trade-off discussion: Huffman is optimal for known per-symbol probability but not for adaptive / streaming inputs.

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

STUDY HERE: topic 3, Trees and Tries

HARD | IN APP LC 834 | PracHub | Technical Screen | 21 of 22

Compute distance-sum from every tree node

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 123/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #834 Sum of Distances in Tree

LeetCode match: LC 834 Sum of Distances in Tree

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

QUESTION

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

APPROACH

Rerooting DP: one DFS computes subtree sizes and the root's distance sum, a second DFS derives each child as ans[child] = ans[parent] - size[child] + (n - size[child]). O(n).

HOW TO THINK ABOUT IT

Computing the sum of distances from each node separately costs O(n) per node. Instead compute it once for the root, then derive the neighbour's answer in O(1): moving the root from a parent to its child makes every node in the child's subtree one step closer and every other node one step farther. That needs only the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree (re-rooting dynamic programming).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = []
    for node in range(n):
        dist = bfs(adj, node)
        ans.append(sum(dist))
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

OPTIMAL SOLUTION · Re-rooting with subtree sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

FOLLOW-UP 2 The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

FOLLOW-UP 3 Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance-sum from every tree node
#
# This question evaluates understanding of tree and graph
# algorithms, including computation of pairwise distances
# and techniques for aggregating path lengths across
# nodes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Find the node with the smallest distance sum (the
# tree's centroid).
# 2. The tree has weighted edges.
# 3. Maximum distance (eccentricity) from each node
# instead of the sum.
#
# INTERVIEW NOTES
# Computing the sum of distances from each node separately
# costs O(n) per node. Instead compute it once for the
# root, then derive the neighbour's answer in O(1): moving
# the root from a parent to its child makes every node in
# the child's subtree one step closer and every other node
# one step farther. That needs only the subtree sizes.
#
```

```python
# BRUTE FORCE (say it first): BFS from every node
# For each node run a BFS and add up the distances.
# Correct and the oracle; n searches of O(n) each. Time
# O(n^2). Space O(n).


from collections import defaultdict
from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def distance_sums_brute(n, edges):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    out = []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
```

```python
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
```

```python
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans


# FOLLOW-UP 1: Find the node with the smallest distance
# sum (the tree's centroid).
# Answer: The answer array above gives it directly: argmin
# of ans. Equivalent: the centroid, found by walking
# toward the heavier subtree.
def best_center(n, edges):
    # the node with the smallest distance sum
    sums = distance_sums(n, edges)
    return sums.index(min(sums))


# FOLLOW-UP 3: Maximum distance (eccentricity) from each
# node instead of the sum.
# Answer: The farthest node from any node is one end of a
# diameter; compute distances from both diameter ends and
# take the maximum per node. O(n).
def eccentricities(n, edges):
    # the farthest node from anywhere is an end of a
    # diameter
    adj = build(n, edges)
    d0 = distances_from(adj, 0)
    a = max(d0, key=d0.get)
    da = distances_from(adj, a)
    b = max(da, key=da.get)
    db = distances_from(adj, b)
    return [max(da[v], db[v]) for v in range(n)]


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
```

```python
    assert distance_sums(6, edges) == want
    assert best_center(6, edges) == 2
    assert distance_sums(1, []) == [0]
    # weighted: the path 0 -(2)- 1 -(3)- 2
    w = [[0, 1, 2], [1, 2, 3]]
    assert distance_sums(3, w) == [7, 5, 8]
    # eccentricity: the farthest node from each node
    assert eccentricities(6, edges) == [2, 3, 2, 3, 3, 3]
```

FUNCTION AND TEST CASES

Signature

```python
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

Sample call

```python
solution(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[0]`
`(4, [[0, 1], [1, 2], [1, 3]])`	`[5, 3, 5, 5]`

ALSO REPORTED · 1 more

Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

STUDY HERE: topic 3, Trees and Tries

HARD | IN APP LC 648 | PracHub | Technical Screen | 22 of 22

Implement Longest-Match Text Replacement

POSTED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TALK ALL TIME 31.9 30 DAYS 0 PRIORITY 120/228

ABOUT THIS QUESTION

Topic: Trees and Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #648 Replace Words

LeetCode match: LC 648 Replace Words / LC 833 Find And Replace in String

Posted on PracHub: 2026-01-20 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/implement-longest-match-text-replacement

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed.

Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2",
 "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.

QUESTION

2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy":
    "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

APPROACH

Build a trie of tokens; scan the text and at each position walk the trie, remembering the longest terminal match; emit its id and jump ahead, else copy the char. Aho-Corasick if the text is large.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary token starting there. Checking every token costs the size of the dictionary per character, so share prefixes in a trie: walk the text from the current position down the trie, remember the last node that ends a token, and stop when the trie has no further edge. The remembered node is the longest match.

Basis: LeetCode 648 (Replace Words) and the longest-prefix matching used in tokenizers and routing tables. Scan left to right, at each position take the longest dictionary token that matches, else copy the character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def replace(text, entries):
    out, i = [], 0
    while i < len(text):
        best = ""
        for token in entries:
            if text.startswith(token, i):
                if len(token) > len(best):
                    best = token
        if best:
            out.append(entries[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out
```

TIME	SPACE
O(len(text) * tokens * token length)	O(1) extra

WHY IT IS SLOW

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

OPTIMAL SOLUTION · Trie walk that remembers the last token end

TIME	SPACE
O(len(text) * longest token)	O(total token length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

FOLLOW-UP 2 The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

FOLLOW-UP 3 Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. **Cost:** still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Longest-Match Text Replacement
#
# You are given a text string and a dictionary of
# replacement entries. Each dictionary entry has the form
# token:id, where token is a string pattern and id is the
# value that should be emitted when that token is matched.
# Implement a function that scans the text from left to
# right and produces an output sequence using greedy
# longest-prefix matching: 1. At the current text
# position, find all dictionary tokens that match the
# substring starting at that position. 2. If at least one
# token matches, choose the longest matching token, append
# its corresponding id to the output, and advance by the
# length of that token. 3. If no token matches, append the
# original character at the current position to the output
# and advance by one character. 4. Continue until the
# entire text has been processed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority is not longest match: insertion order, a
# score, or a custom comparator.
# 2. The text is huge or the dictionary changes often.
# 3. Overlapping matches or word boundaries.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary token
# starting there. Checking every token costs the size of
# the dictionary per character, so share prefixes in a
# trie: walk the text from the current position down the
# trie, remember the last node that ends a token, and stop
# when the trie has no further edge. The remembered node
# is the longest match.
#
# BRUTE FORCE (say it first): At every position try every
# dictionary token
```

```
# For each position test startswith for every token and
# keep the longest. Correct and the oracle; each position
# checks the entire dictionary. Time O(len(text) * tokens
# * token length). Space O(1) extra.


# BRUTE FORCE (say it first): At every position try every
# dictionary token | time O(len(text) * tokens * token
# length)
def replace_longest_brute(text, mapping):
    out, i = [], 0
    while i < len(text):
        best = None
        for tok in mapping:
            if text.startswith(tok, i) and (
                best is None or len(tok) > len(best)
            ):
                best = tok
        if best:
            out.append(mapping[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
# FOLLOW-UP 1: The priority is not longest match:
# insertion order, a score, or a custom comparator.
# Answer: Collect every token that matches at the position
# (walk the trie and gather all token ends along the path,
# not just the last) and choose with the comparator. The
# trie gives the candidates; the policy is separate.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out

    # FOLLOW-UP 1: choose= picks by priority, not length
    def replace(self, text, choose=None,
                word_boundary=False):
        # choose(candidates) picks the winner; the default
        # is the
        # longest token
        choose = choose or (lambda cs: cs[-1])
        out, i = [], 0
        while i < len(text):
            found = self.candidates(text, i)
            # FOLLOW-UP 3: only whole-word matches
            if word_boundary:
                found = [c for c in found if
                         self.edge(text, i, len(c[0]))]
            if found:
                token, ident = choose(found)
                out.append(ident)
                i += len(token)
            else:
                # nothing matches here: keep the character
                out.append(text[i])
                i += 1
        return out

    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    r = Replacer(table)
    # abc beats ab at position 0
    assert r.replace("abcab") == [2, 1]
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    # priority by insertion order instead of length
    order = list(table)
    first = r.replace("abc", lambda cs: min(
        cs, key=lambda c: order.index(c[0])))
    assert first == [1, 3]
    # whole words only
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(text: str, dictionary: dict[str, str]) -> lis
t[str]:
```

Sample call

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

Test cases

Input	Expected
("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})	["2", "d", "3", "z"]
("aaaaa", {"a": "X", "aa": "Y"})	["Y", "Y", "X"]

ALSO REPORTED · 1 more

Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.

Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.

Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).

If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.